"Roads? Where we're going, we don't need roads.“

Doc Brown (Christopher Lloyd)

#WPNinjaCH

Automatic Deployment Of Azure Virtual Desktop

AVD as Code

Thank You Sponsors

#WPNinjaCH

Microsoft

baseVISION
SECURE & MODERN ENDPOINT MANAGEMENT

EPIC FUSION
BRING IT ALL TOGETHER

Agenda

#WPNinjaCH

Key takeaways:

- **Understand the need for automation of AVD**
- **Decision-making on IaC-tool for AVD automation**
- **Deep dive in our approach**
- **20 minutes and an AVD environment can be ready to use**

Introduction to AVD as Code

Get to know Michele and Flavio and today's topic

Infrastructure as Code tool for AVD

Which Infrastructure as Code tool you should choose

Lifecycle of AVD as Code

Considerations regarding the lifecycle of AVD as Code

Deep dive & live demo

Deep dive into our use case with a live demo

Closing of the speech

Q&A, session feedback, and socials

Speaker - Michele

.\tech

- Cloud Engineer @ TurnKey Services AG
- MCT and former Microsoft Learn Student Ambassador (2024)
- More than 10 Microsoft Certifications (Expert, Associate, Speciality und Fundamentals)

.\private

- Swiss / Italian; Gym, cars / motorcycle, friends and family, …

.\socials

linkedin.com/in/michele.blum

michele.blum@duo-infernale.ch

duo-infernale.ch

github.com/Quattro99

Speaker - Flavio

.\tech

- Public Cloud Engineer @Inventx AG (Banking & Insurance)
- Former Microsoft Learn Student Ambassador (2024)
- 9x Microsoft Certifications (Expert, Associate, Speciality und Fundamentals)

.\private

- Swiss; Racket sport, Travel

.\socials

@flaviomeyer.bsky.social

linkedin.com/in/flaviomeyer

flavio.meyer@duo-infernale.ch

duo-infernale.ch

github.com/flaviomeyer

Introduction To AVD as Code

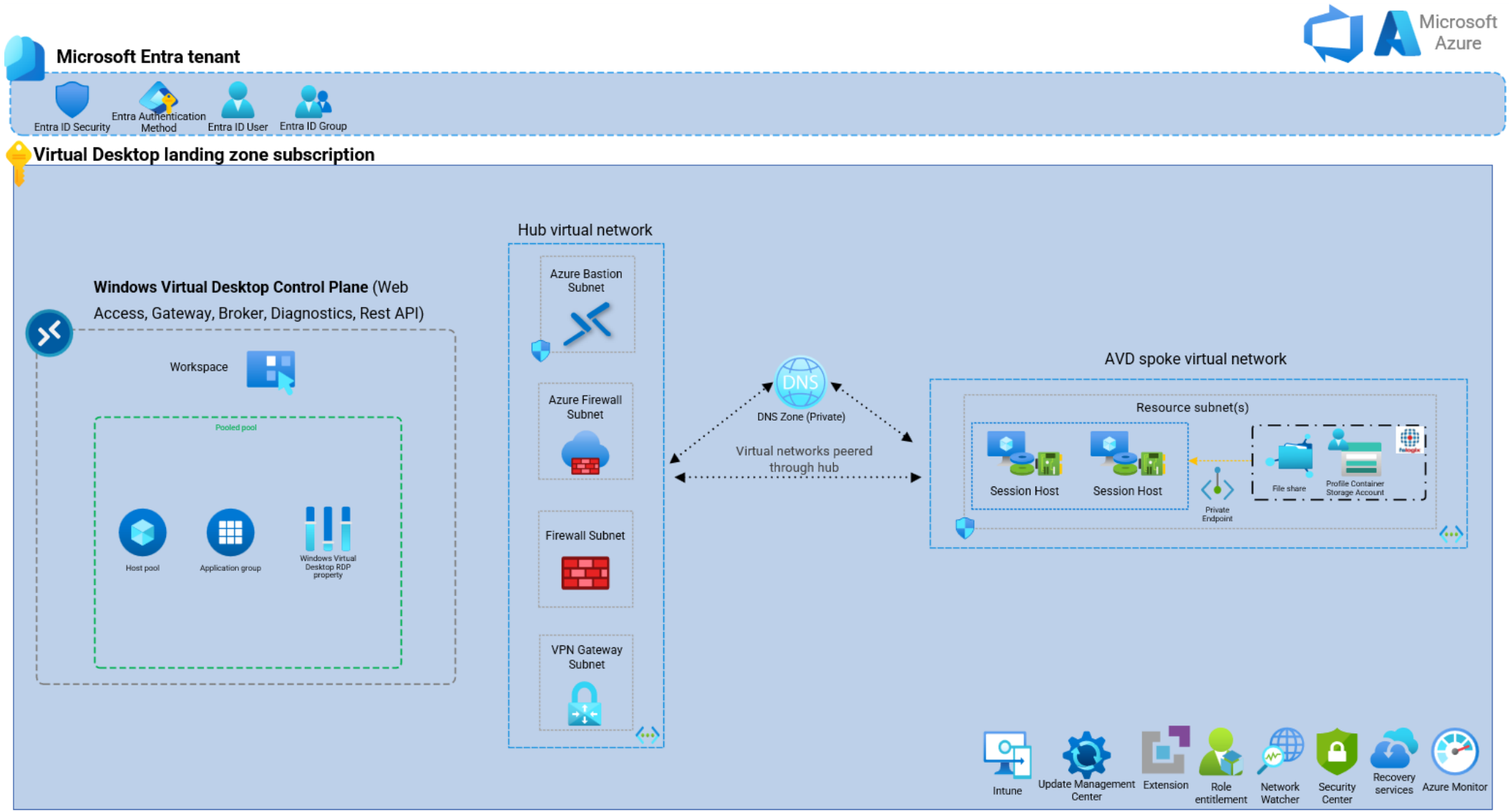
Understanding AVD And Automation

#WPNinjaCH

- Remote Desktop Services in the Azure Cloud
- Desktop and application virtualization
- In theory a free Azure service
- Automation leverages efficiency and prevents human errors
- Rectify any configuration drifts

The Use Case

#WPNinjaCH



Which Infrastructure as Code Tool You Should Choose

SWOT Analysis Of Terraform

Internal analysis	**Strengths** • Multi-cloud scenarios • Support for multiple service providers • Scalability	**Weaknesses** • Analysing error messages is not always understandable • Performance issues

SWOT Analysis Of Terraform

#WPNinjaCH

External analysis	Opportunities	Threats
	• Large community • Broad market acceptance for various application areas	• Acquisition by IBM • Rapid innovations in cloud technology

SWOT Analysis Of Bicep

Internal analysis	**Strengths** • In-depth Azure integration • Simple syntax • Speed compared to Terraform during deployment	**Weaknesses** • Vendor lock-in • Relatively small community

SWOT Analysis Of Bicep

External analysis	Opportunities	Threats
	• Actively improved by Microsoft • Integration within the Microsoft ecosystem	• Competition with other IaC tools (Terraform) • Limitations with multi-cloud providers

Considerations Regarding The Lifecycle Of AVD as Code

General Best Practices

#WPNinjaCH

Steer clear of preview features

Patch often, and fast

Security and Compliance

Automation and scaling

Infrastructure As Code Template

#WPNinjaCH

- Usage of Azure DevOps and Azure Bicep
- ***main.bicep*** contains 489 lines of code
- Without the modulization approx. 1800 lines of code

```
// main.bicep //
module vnethub 'network/vnet.bicep' = {}

// Modul vnet.bicep //
resource vnet 'Microsoft.Network/virtualNetworks@2023-11-01' = {}
```

Host Images

- **Marketplace images**: Reliability, less maintenance (use case)
- **Modular updates**: Image SKUs, Windows 11 multi session
- **Dynamic selection**: DevOps parameters, flexibility
- **OS actuality**: "latest" tag, off-hours updates with MEM

```
// Publisher of the session host image
@description('The image publisher for the session host.')
param sessiohostImageReferencePublisher string = 'microsoftwindowsdesktop'

// Offer of the session host image
@description('The image offer for the session host.')
param sessionhostImageReferenceOffer string = 'office-365'

// SKU of the session host image
@description('The image SKU for the session host.')
param sessionhostImageReferenceSku string = 'win11-25h2-avd-m365'

// Version of the session host image
@description('The image version for the session host.')
param sessionhostImageReferenceVersion string = 'latest'
```

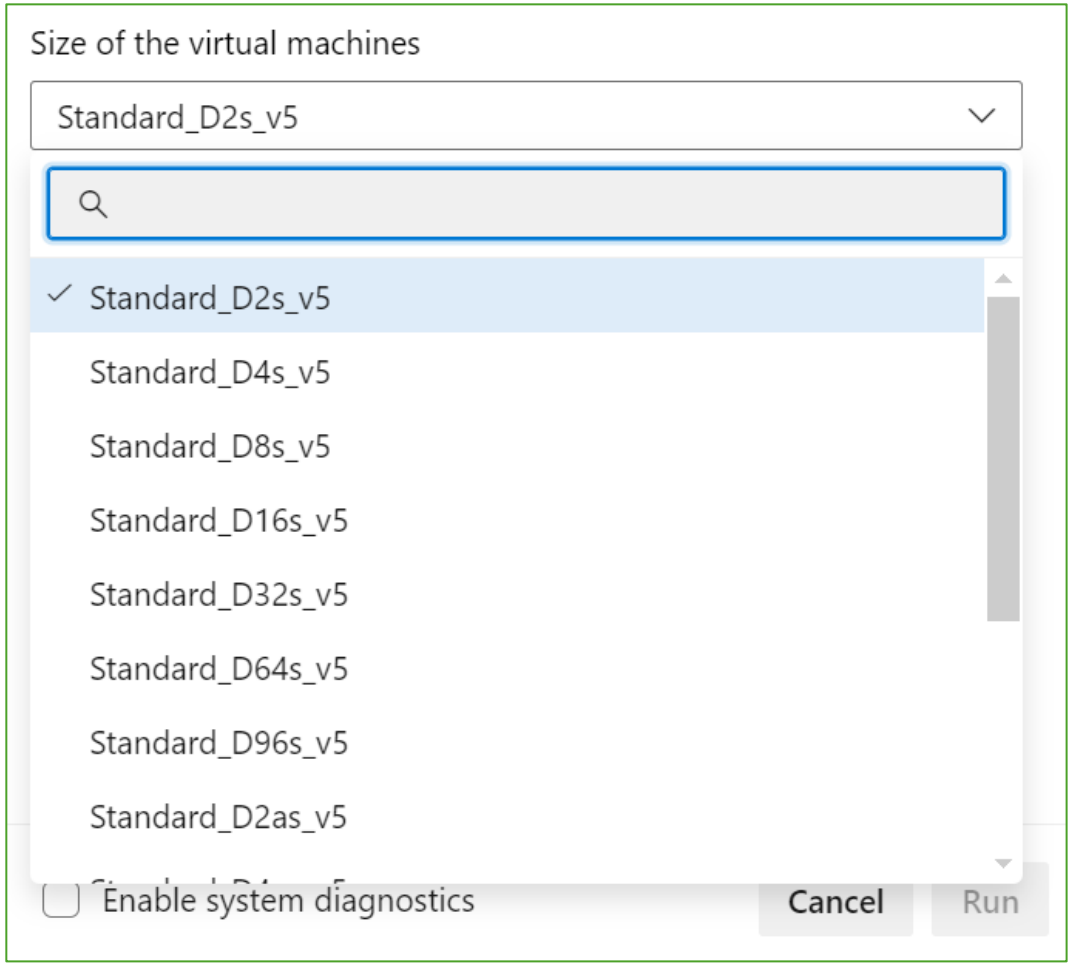
AVD Agent

#WPNinjaCH

```
resource vdpool 'Microsoft.DesktopVirtualization/hostPools@2023-09-05' = {
    agentUpdate: {
      maintenanceWindows: [
        {
          dayOfWeek: 'Friday'
          hour: 20
        }
      ]
      maintenanceWindowTimeZone: 'W. Europe Standard Time'
      type: 'Scheduled'
      useSessionHostLocalTime: true
  }
}
```

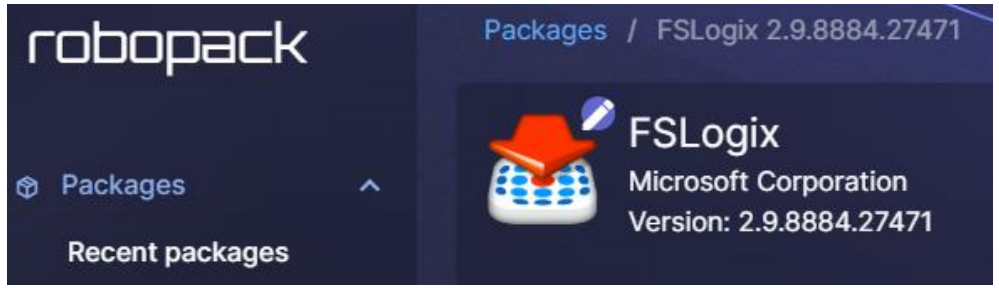
VM Generations

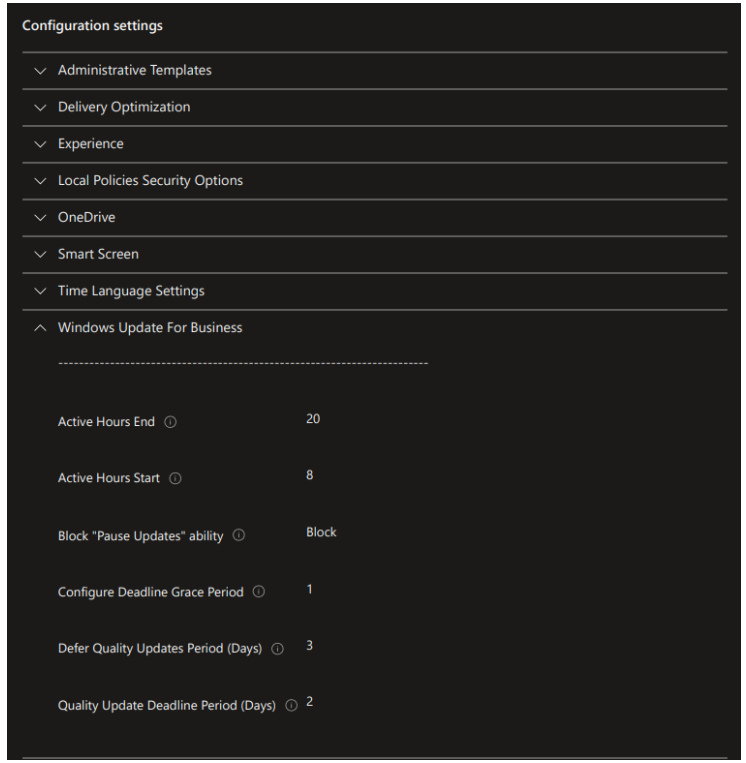
- Microsoft offers a variety of virtual machines
- Our favourite: D series for AVD
- Update your “standard” or “go-to” VM-series to newer generation from time to time
- You can choose, which VM you want in our Pipeline (more in the live demo)

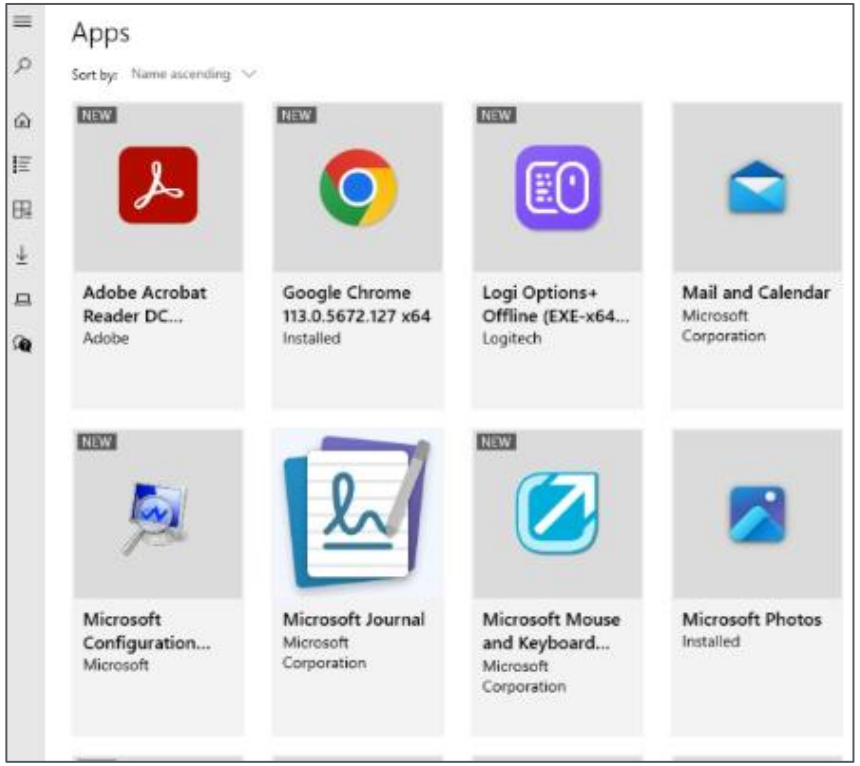


Operating System & Application

#WPNinjaCH







Deep Dive & Live Demo

#WPNinjaCH

Azure DevOps + Azure Virtual Desktop

Pipeline And Bicep Flow

#WPNinjaCH

Pipeline YAML (Customer) - Params

#WPNinjaCH

```yaml
##### ---------------------- PARAMETERS ---------------------- #####

# Define parameters for the pipeline

parameters:

  - name: customerAbbreviation

    displayName: "Customer Abbreviation (3 characters max, no special characters and no capital letters)"

    type: string

  - name: environment

    displayName: "Environment for Resource Deployment (e.g., Production, Test, Development)"

    type: string

    default: prod

    values:

      - "prod"

      - "test"

      - "dev«

...

...
```

Pipeline YAML (Customer) - Stages

#WPNinjaCH

```yaml
##### ---------------------- STAGES ---------------------- #####

# Include the static pipeline as a template

stages:

  - template: /_Pipelines\_Templates\AVD\Full landing zone\Entra ID authentication only\Pooled session hosts\tmpl-prod-
mainpipeline-avdac-001.yml

    parameters:

      customerAbbreviation: ${{ parameters.customerAbbreviation }}

      environment: ${{ parameters.environment }}

      location: ${{ parameters.location }}

      locationAVD: ${{ parameters.locationAVD }}

      vdwsName: ${{ parameters.vdwsName }}

      tags: ${{ parameters.tags }}

      networkId: ${{ parameters.networkId }}

      fsshareQuota: ${{ parameters.fsshareQuota }}

      vmCount: ${{ parameters.vmCount }}

      vmSize: ${{ parameters.vmSize }}

...

...
```

Pipeline YAML (Template) - Stage Lint

#WPNinjaCH

```yaml
##### ---------------------- STAGES ---------------------- #####

stages:

    - stage: Lint # Lint stage for checking the Bicep template syntax

    jobs:

      - job: LintCode

        steps:

          - checkout: self # Check out the repository

          - task: AzureCLI@2

            inputs:

              azureSubscription: ${{ parameters.serviceConnection }} # Service connection for Azure subscription

              scriptType: "pscore"

              scriptLocation: "inlineScript"

              inlineScript: |

                az bicep build --file '${{ parameters.templateFile }}'
```

Pipeline YAML (Template) - Stage Validate

#WPNinjaCH

```yaml
- stage: Validate # Validate stage for validating the Bicep template
  dependsOn: Lint
  jobs:
    - job: ValidateBicepTemplate
      steps:
        - checkout: self # Check out the repository
        - task: AzureCLI@2
          inputs:
            azureSubscription: ${{ parameters.serviceConnection }} # Service connection for Azure subscription
            scriptType: "pscore"
            scriptLocation: "inlineScript"
            inlineScript: |
              az deployment sub validate --location '${{ parameters.location }}' --template-file '${{ parameters.templateFile }}' --
              parameters gpar_environment='${{ parameters.environment }}' dpar_location='${{ parameters.location }}' dpar_locationAVD='${{
              parameters.locationAVD }}' gpar_customerAbbreviation='${{ parameters.customerAbbreviation }}' gpar_tags='${{
              convertToJson(parameters.tags) }}' dpar_networkId='${{ parameters.networkId }}' dpar_vdwsName='${{ parameters.vdwsName }}'
              dpar_fsshareQuota='${{ parameters.fsshareQuota }}' dpar_vmCount='${{ parameters.vmCount }}' dpar_vmSize='${{ parameters.vmSize
              }}' dpar_privateDNSName='${{ parameters.privateDNSName }}' dpar_vmlocaladminName='${{ parameters.localadminName }}'
              dpar_vmlocaladminPassword='${{ parameters.localadminPassword }}' dpar_rbacObjectIdFullDesktopUsers='${{
              parameters.rbacObjectIdFullDesktopUsers }}' dpar_rbacObjectIdRBACAVDAdmin='${{ parameters.rbacObjectIdRBACAVDAdmin }}'
              dpar_rbacObjectIdRBACAVDUsers='${{ parameters.rbacObjectIdRBACAVDUsers }}'
```

Azure DevOps Pipeline YAML (Template) - 2

#WPNinjaCH

```yaml
  - stage: Preview  # Preview stage for previewing the changes before deployment
    dependsOn: Validate
    jobs:
      - job: Preview
        steps:
          - checkout: self # Check out the repository
          - task: AzureCLI@2
            inputs:
              azureSubscription: ${{ parameters.serviceConnection }}
              scriptType: "pscore"
              scriptLocation: "inlineScript"
              inlineScript: |
                az deployment sub what-if --location '${{ parameters.location }}' --template-file '${{ parameters.templateFile }}' --parameters gpar_environment='${{ parameters.environment }}' dpar_location='${{ parameters.location }}' dpar_locationAVD='${{ parameters.locationAVD }}' gpar_customerAbbreviation='${{ parameters.customerAbbreviation }}' gpar_tags='${{ convertToJson(parameters.tags) }}' dpar_networkId='${{ parameters.networkId }}' dpar_vdwsName='${{ parameters.vdwsName }}' dpar_fsshareQuota='${{ parameters.fsshareQuota }}' dpar_vmCount='${{ parameters.vmCount }}' dpar_vmSize='${{ parameters.vmSize }}' dpar_privateDNSName='${{ parameters.privateDNSName }}' dpar_vmlocaladminName='${{ parameters.localadminName }}' dpar_vmlocaladminPassword='${{ parameters.localadminPassword }}' dpar_rbacObjectIdFullDesktopUsers='${{ parameters.rbacObjectIdFullDesktopUsers }}' dpar_rbacObjectIdRBACAVDAdmin='${{ parameters.rbacObjectIdRBACAVDAdmin }}' dpar_rbacObjectIdRBACAVDUsers='${{ parameters.rbacObjectIdRBACAVDUsers }}'
```

```yaml
- job: waitForValidation # Manual validation step
    displayName: Wait for Preview validation
    dependsOn: Preview
    pool: server
    steps:
      - task: ManualValidation@0
        inputs:
          instructions: "Please validate the Preview stage and resume"
          onTimeout: "resume"
```

Azure DevOps Pipeline YAML (Template) - 2

#WPNinjaCH

```yaml
- stage: Deploy   # Deploy stage for deploying the resources
  dependsOn: Preview
  jobs:
    - job: Deploy
      steps:
        - checkout: self # Check out the repository
        - task: AzureCLI@2
          continueOnError: true # Continue on error to allow manual validation
          inputs:
            azureSubscription: ${{ parameters.serviceConnection }} # Service connection for Azure subscription
            scriptType: "pscore"
            scriptLocation: "inlineScript"
            inlineScript: |
              az bicep install

              az deployment sub create --location '${{ parameters.location }}' --template-file '${{ parameters.templateFile }}' --parameters gpar_environment='${{ parameters.environment }}' dpar_location='${{
              parameters.location }}' dpar_locationAVD='${{ parameters.locationAVD }}' gpar_customerAbbreviation='${{ parameters.customerAbbreviation }}' gpar_tags='${{ convertToJson(parameters.tags) }}'
              dpar_networkId='${{ parameters.networkId }}' dpar_vdwsName='${{ parameters.vdwsName }}' dpar_fsshareQuota='${{ parameters.fsshareQuota }}' dpar_vmCount='${{ parameters.vmCount }}' dpar_vmSize='${{
              parameters.vmSize }}' dpar_privateDNSName='${{ parameters.privateDNSName }}' dpar_vmlocaladminName='${{ parameters.localadminName }}' dpar_vmlocaladminPassword='${{ parameters.localadminPassword }}'
              dpar_rbacObjectIdFullDesktopUsers='${{ parameters.rbacObjectIdFullDesktopUsers }}' dpar_rbacObjectIdRBACAVDAdmin='${{ parameters.rbacObjectIdRBACAVDAdmin }}' dpar_rbacObjectIdRBACAVDUsers='${{
              parameters.rbacObjectIdRBACAVDUsers }}'
```

main.bicep

#WPNinjaCH

```
//// ---------------------- SCOPE ---------------------- /////

targetScope = 'subscription’


///// ---------------------- PARAMETERS -------------------- /////

// The abbreviation of the customer for the deployment.

param gpar_customerAbbreviation string

// The environment for the deployment (e.g., dev, test, prod).

param gpar_environment string

// Tags to be applied to the resources for resource management and
organization.

param gpar_tags object


///// ---------------------- VARIABLES -------------------- /////

var var_tags = gpar_tags != {}

  ? gpar_tags

  : {

      Environment: gpar_environment != '' ? gpar_environment :
'prod'

      Customer: gpar_customerAbbreviation != '' ?
gpar_customerAbbreviation : 'duo'
```

```
///// ---------------------- MODULES ----------------------
/////

// Defines the AVD resource group.

module rgavd '_Modules/resourcegroup/rg.bicep' = {

  name: var_rgAVDName

  params: {

    rgName: var_rgAVDName

    rgLocation: dpar_location

    rgTags: var_tags

  }

}

// Defines the storage resource group.

module rgstorage '_Modules/resourcegroup/rg.bicep' = {

  name: var_rgStorageName

  params: {

    rgName: var_rgStorageName

    rgLocation: dpar_location

    rgTags: var_tags

  }
```

module-x.bicep

#WPNinjaCH

```
///// ---------------------- PARAMETERS ----------------------
/////

// The name of the Virtual Desktop Host Pool

@description('The name of the Virtual Desktop Host Pool')

param vdpoolName string


// The name of the VM being created

@description('The name of the virtual machine.')

param vmName string


// The location where the session host will be deployed

@description('The location of the session host.')

param vmLocation string


// Tags to be applied to the session host

@description('Tags to be applied to the session host.')

param vmTags object
```

```
///// ---------------------- RESOURCES ----------------------
/////

// Define the existing 'Virtual Desktop Host Pool' resource

resource vdpool
'Microsoft.DesktopVirtualization/hostPools@2024-04-03'
existing = {

  name: vdpoolName

}

// Create a network interface for the session host

resource nic 'Microsoft.Network/networkInterfaces@2024-05-01'
= {

  name: vmNicName

  location: vmLocation

  tags: vmTags

  properties: {

    ipConfigurations: [

      {

        name: vmNicIpConfigurationName

        properties: {

          subnet: { id: snetSpokeAVDId
```

AVD Deployment

#WPNinjaCH

Azure DevOps duo-infernale / AVD / Pipelines

Search

AVD

Overview

Boards

Repos

Pipelines

Pipelines

Environments

Library

Test Plans

Artifacts

Project settings

Pipelines

New pipeline

Recent All Runs

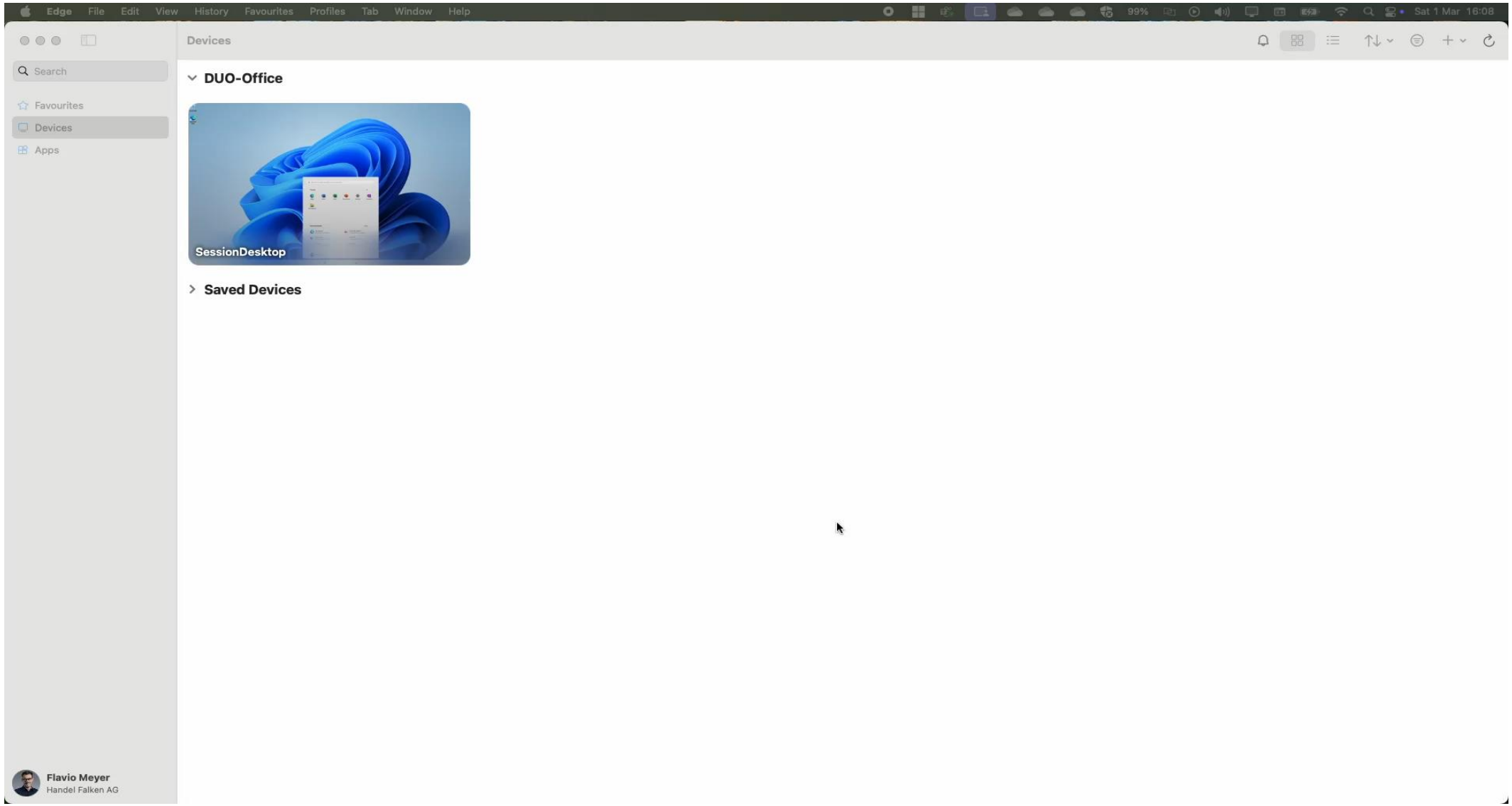
Filter pipelines

Recently run pipelines

Pipeline	Last run
AVD - PROD - Duo Infernale	No runs yet

AVD End User Experience

#WPNinjaCH



#WPNinjaCH

MAR-19-2026 13:40

Are we back in time?

Summary

I. We learned why automating AVD is crucial for reliable, scalable, and cost-efficient deployments.

II. We understood how to evaluate and select the right Infrastructure-as-Code tool, and why Bicep was the best fit for our use case.

III. We explored our AVD automation approach.

IV. We experienced a complete AVD deployment in action. From code to a fully functional virtual desktop environment in under 20 minutes.

Thank you

Questions And Answers

#WPNinjaCH

Session Feedback

#WPNinjaCH

Automatic deployment of Azure Virtual Desktop

We love Feedback

Github Repository & Socials

#WPNinjaCH

https://github.com/DuoInfernale/AVDaC

https://duo-infernale.ch

https://www.linkedin.com/company/duoinfernale

https://www.instagram.com/duoinfernale_/